

Diagram-Aware Automatic Review of Software Design Documents Using Multimodal Large Language Models

Takasaburo Fukuda
Fujitsu Ltd.
Kawasaki, Japan
f.takasaburo@fujitsu.com

Susumu Tokumoto
Fujitsu Ltd.
Kawasaki, Japan
tokumoto.susumu@fujitsu.com

Abstract—With the growing capabilities of Large Language Models (LLMs), their application to software engineering tasks has garnered increasing attention. While our previous work focused on automated review of software design documents composed of text and tables, diagram-based artifacts such as UML or screen transition diagrams remained outside the scope of analysis. Reviewing such documents requires accurate interpretation of structural and semantic elements, including nodes, edges, and transition conditions.

This paper proposes a hybrid diagram-understanding method that leverages multimodal LLMs together with diagram structure extracted from Office Open XML (OOXML) representations, guided by Chain-of-Thought prompting, to incrementally interpret diagrams in real-world design documents. The method enables integrated analysis across diagrams, tables, and textual content within the review process. Evaluation results show that the proposed approach achieves high accuracy in tasks involving structural recognition, such as consistency checking. However, the results also highlight persistent challenges in semantic-level review tasks, particularly in interpreting complex transition conditions. These findings provide insights into the current capabilities and limitations of multimodal LLMs for high-stakes design document reviews.

Index Terms—Software Design Review, Large Language Models, Diagram Understanding

I. INTRODUCTION

Recent advancements in Large Language Models (LLMs) have sparked growing interest in their application to software engineering. Although LLMs have demonstrated remarkable success in code generation tasks [1], their use in upstream activities, especially in the review of software design documents, remains underexplored. In industrial software development, design documents serve as essential communication artifacts among stakeholders, including clients and system engineers. Defects in these documents often propagate into downstream phases, leading to implementation omissions, inconsistent behaviors, and costly rework. [2] [3] [4] As a result, design reviews constitute a critical and labor-intensive quality assurance activity.

Our earlier work proposed an automatic review method for design documents consisting of textual and tabular descriptions [5]. However, real-world design artifacts frequently include

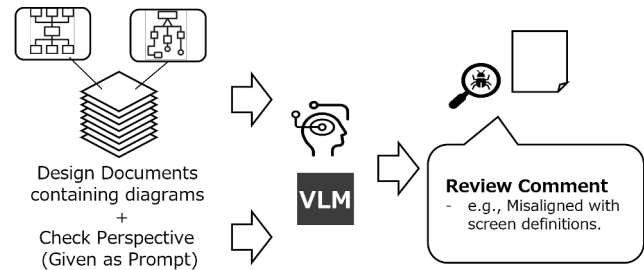


Fig. 1. Illustration of Multimodal LLM Utilization in the Review Process

graph-structured UML-style diagrams, including screen transition diagrams and business process flows. These diagrams encode functional behavior, UI flow, and system structure in graph-based formats, and thus must be interpreted accurately to support comprehensive review.

While recent multimodal LLMs such as GPT-4V and GPT-4o support image input, our initial experiments indicated that raw image-based diagram comprehension is often insufficient for high-stakes review tasks. Complex diagrams require precise recognition of node labels, edge relationships, and layout semantics to map visual elements to textual counterparts. These capabilities remain challenging for current vision-language models when applied to real-world design artifacts that vary in style, density, and layout conventions.

To address these challenges, we first analyze the nature of industrial review tasks and how they depend on different forms of diagram understanding. In this study, we focus on diagrams that can be represented as graphs, including screen transition diagrams and business process flows. These types of diagrams account for the majority of visual content found in software design documents, such as those based on UML notations. In contrast, we exclude artifacts such as screen layout mockups or UI wireframes, which involve spatial composition rather than logical structure. This scope reflects the current limitations of multimodal LLMs in handling visually dense or non-structured images.

Furthermore, the target design documents are drawn from

real-world Japanese software projects, where Microsoft Excel is the predominant format for authoring specifications. As noted in prior work, over half of Japanese software design documents are created in Excel due to its flexibility for managing structured tabular information [6]. In this context, diagrams are often embedded as shape-based elements conforming to the Office Open XML (OOXML) format [7]. We extract diagrammatic structure from these Excel-based OOXML representations, incorporating bounding boxes, textual labels, and shape identifiers as auxiliary signals to guide multimodal LLMs. These enhancements reduce structural hallucination, improve label fidelity, and strengthen grounding between diagrams and textual descriptions.

Finally, we evaluate the proposed method using actual design documents and review tasks collected from an enterprise software development project. By mapping review tasks to their required levels of diagram comprehension and comparing LLM performance across task types, we demonstrate the practical boundary of applicability for current multimodal LLMs. These findings offer insight into where LLM-based review can already provide value and where additional advances are required for broader adoption.

This paper aims to extend LLM-based review support to such diagram-inclusive design documents. Our contributions are threefold:

- **Organization of industrial review tasks:** We analyze real-world design review perspectives and clarify the forms of diagram comprehension required for each task. This task-oriented decomposition provides a structured basis for determining what levels of recognition (e.g., label identification, structural understanding, semantic interpretation) are necessary to support meaningful automation.
- **Hybrid diagram-aware review method with OOXML:** We propose a hybrid method that combines multimodal LLMs with structural information extracted from OOXML representations. By augmenting visual inputs with coordinate-level and label-level features, the method improves the accuracy and stability of diagram interpretation in Excel-based design documents used in industrial software development.
- **Empirical evaluation on real design documents:** We conduct an extensive evaluation using actual review tasks and design documents from enterprise-scale software development. This evaluation reveals which categories of review tasks current multimodal LLMs can realistically support and clarifies the practical limitations that remain due to semantic complexity and diagram variability.

The remainder of this paper is organized as follows. Section 2 reviews related work on large language models for software design review and recent advances in diagram understanding within multimodal vision-language models. Section 3 provides a task-oriented analysis of industrial design review activities, mapping each review perspective to the corresponding level of diagram understanding required for automation. Section

4 presents our proposed hybrid method, which integrates multimodal LLMs with structural information extracted from OOXML to improve diagram interpretation. Section 5 reports on an experimental evaluation using real-world design documents, examining both recognition accuracy and the practical applicability of the proposed method.

II. RELATED WORK

A. Large Language Models for Software Design Review

Design review is a critical quality assurance activity in software development, intended to verify the consistency, completeness, and correctness of design artifacts prior to implementation. International standards such as IEEE 1028-2008 [8] and ISO/IEC 20246:2017 [9] define structured review methods, including walkthroughs, technical reviews, and inspections. Additionally, techniques such as checklist-based reading (CBR) and scenario-based reading (SBR) are widely employed in industrial practice [10] [11]. An industrial survey involving more than 5,000 projects reported that design reviews typically require 5.5 to 6.4 hours per KSLOC [12], emphasizing the urgent need to improve the efficiency and automation of review processes. Nevertheless, research that directly focuses on the review of design documents remains extremely limited. A systematic literature review by Schmid et al. [13] pointed out that most studies on LLM-based design support concentrate on generative or constructive tasks, such as detecting design patterns or extracting architectural components, with very few addressing design documentation from the viewpoint of review and quality assurance. It has also been reported that only 1.29 percent of LLM-related software engineering studies target the design phase [14].

Meanwhile, a number of studies have explored LLM-based review support for natural language documents. For example, researchers have investigated ambiguity detection and automated comment generation for software requirements specifications (SRS) [15], as well as multi-agent approaches for automating literature reviews [16]. MMReview [17] is another notable effort, which constructed a multimodal benchmark for research paper peer review. Although MMReview focuses on academic papers, it incorporates multimodal inputs, including figures, tables, and PDF images. As such, it provides a useful precedent for examining how LLMs may assist in reviewing documents that contain visual elements. However, its target domain differs significantly from that of design documents in software development, which often include screen transition diagrams, system configurations, and architectural charts governed by domain-specific notations and layout rules.

In parallel, many studies have investigated the use of LLMs for design generation or transformation. These include multi-agent design synthesis using MetaGPT [18], LLM support for UML modeling [19], and code generation from UML diagrams [20]. Although such studies demonstrate the feasibility of LLMs in design-related tasks, their focus is not on quality assurance activities, such as identifying review concerns or checking for consistency across design artifacts.

TABLE I
REVIEW PERSPECTIVES AND REQUIRED DIAGRAM RECOGNITION LEVELS.

Review Perspective	Example Review Item	Required Recognition Level
Sufficiency Check	Whether all operational flows described in the business process diagram are present in the screen transition diagram	Semantic Recognition Level
	Whether conditional branches based on business process outcomes (e.g., approval/rejection) are explicitly represented in the diagram	Semantic Recognition Level
Standards/Regulations Check	Whether screen IDs and UI labels follow project-specific naming conventions	Label Recognition Level
	Whether prohibited or legacy labels appear in any diagram element	Label Recognition Level
Traceability Check	Whether the screen transitions correspond to the operational flow in the requirement specification	Structural Recognition Level
	Whether each user action described in the requirements has a corresponding transition or screen state in the diagram	Structural Recognition Level
Functional Requirements Check	Whether the provided screen functions satisfy the functional requirements	Semantic Recognition Level
	Whether branching logic for each function is accurately reflected in transitions	Semantic Recognition Level
Consistency Check	Whether screen transition diagrams and detailed screen specifications use consistent labels and identifiers	Label Recognition Level
	Whether the hierarchy or linkage of screens is consistently represented between the screen transition diagram and the screen definition tables	Structural Recognition Level
Clarity Check	Whether screen names clearly describe their purpose without vague wording	Label Recognition Level
	Whether transition labels indicate user operations unambiguously	Label Recognition Level
Non-functional Requirements Check	Whether performance constraints (e.g., timeout flows) are reflected in transition design	Semantic Recognition Level
	Whether accessibility-related flows (e.g., keyboard-only operations) are represented in diagrams	Semantic Recognition Level

In conclusion, design document review is an underexplored research area, and current LLM-based approaches do not yet offer comprehensive support for the multimodal design artifacts commonly used in industrial practice.

B. Diagram Understanding in Multimodal Vision-Language Models

Recent advances in multimodal vision-language models (VLMs) have sparked interest in their capacity to interpret diagram-based inputs, yet persistent limitations remain. For instance, recent evaluations of diagram understanding in VLMs show that while these models can detect individual entities in visual representations, they often fail to capture relational structures and spatial layouts, instead relying on memorized associations rather than true diagram reasoning [21]. Similarly, the VGCURE benchmark reveals widespread deficits in graph comprehension and relational reasoning across 14 large VLMs and 22 tasks [22].

In more specialized domains, benchmarks like Poly-ChartQA, which focuses on multilingual chart question answering over 22,000 visualizations, highlight persistent weaknesses in handling diverse encodings, such as non-Latin scripts and complex legends [23]. Likewise, recent studies have explored code generation from diagrammatic inputs. Antal et al. [24] evaluated GPT-4-Vision's performance in translating UML class diagrams into source code and observed that accuracy deteriorated as diagram complexity increased. Similarly, Yu et al. [25] proposed Flow2Code, a system that uses VLMs to convert flowchart images into Python code, yet their evaluation revealed limited precision in geometric recognition and semantic preservation. FlowLearn [26] fur-

ther evaluates flowchart reasoning across multiple VLMs and identifies consistent difficulty in capturing control flow and layout-based logic.

Underlying these challenges are two structural mismatches: (1) diagrams often encode symbolic semantics, such as nodes, edges, or state transitions, that require discrete reasoning; and (2) typical training pipelines for VLMs lack inductive biases conducive to spatial and relational inference. In response, recent research has proposed hybrid approaches that incorporate structural metadata. For example, Shiinoki et al. leverage editable source formats (e.g., OOXML) to bypass direct image recognition bottlenecks [27], while others employ structure-aware contrastive learning to explicitly model visual relationships [28].

Despite this progress, these approaches have not yet been extended to domain-specific industrial artifacts such as UML diagrams, system architecture charts, or structured design flows. This gap motivates our effort to integrate source-level structural metadata into LLM-based review processes tailored to multimodal design documents.

III. REVIEW PERSPECTIVES AND VISUAL RECOGNITION LEVELS IN DESIGN DOCUMENT REVIEW

In an ideal scenario, the use of LLMs for design document review would allow the entire set of documents to be ingested at once, enabling comprehensive feedback across all review perspectives with a single prompt. However, in practice, no general-purpose LLM possesses the expert-level capability required for such review tasks. Furthermore, prompt length constraints limit the amount of contextual and domain-specific

design knowledge that can be supplied at once, making it impractical to cover all perspectives simultaneously.

To address these constraints, our previous work proposed a classification of eleven distinct review perspectives for design documents [5]. These perspectives cover essential aspects of software design evaluation, including requirement completeness, specification consistency, and UI design adequacy. In the present study, we additionally use these perspectives to identify which areas require expert-level judgment and which might be more amenable to LLM-based support.

However, the original formulation of these perspectives assumes natural language-based design artifacts and does not directly accommodate visual elements such as diagrams. Reviewing diagrammatic elements—such as screen transition diagrams or architectural layouts—requires the model to incrementally extract information encoded visually, including node labels, structural relationships, and inter-node conditions.

Our preliminary experiments, in line with prior work (see Section 2.2), suggest that VLMs exhibit varying levels of difficulty in processing such visual elements, depending on the type of information being recognized. Specifically, visual recognition difficulty can be stratified across the following levels:

- **Level 1: Label Recognition**

The model can reliably extract textual labels attached to nodes or edges. This capability enables basic review tasks such as checking terminology consistency, matching screen or component names, and verifying the presence or absence of specific design elements.

- **Level 2: Structural Recognition**

The model can identify the connectivity and spatial arrangement of nodes. This level supports more advanced review activities, including validation of transition patterns, detection of unreachable or isolated nodes, and verification of structural alignment between diagrams and textual specifications.

- **Level 3: Semantic Recognition**

The model can infer contextual meaning, such as flow control, conditional branching, and procedural relationships. This enables high-level review tasks that require semantic alignment, for example verifying whether business workflows are correctly reflected in screen transition diagrams or whether conditional logic is implemented consistently.

This level-based classification is informed by observed limitations in benchmark evaluations of VLMs, such as FlowLearn’s difficulty with conditional reasoning and Poly-ChartQA’s failures in interpreting complex legends. It also serves as a practical guide to the feasible scope of LLM-assisted reviews depending on visual recognition fidelity.

To clarify how diagram understanding is required in real-world design reviews, we collected actual review items from past software development projects and organized them according to the eleven established review perspectives. Each item was analyzed to determine the minimum level of diagram recognition—such as label identification, structural parsing, or

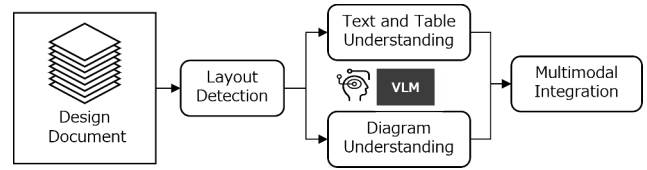


Fig. 2. Overall system architecture.

semantic interpretation—needed to support reliable automation. Table 1 summarizes representative review items for each perspective, with a particular focus on those applicable to diagram-inclusive design documents such as screen transition diagrams and business process flows. This selection reflects the types of diagrams used in the empirical evaluation described later in this paper.

For example, under the “Consistency” perspective, verifying that the hierarchy or linkage of screens is consistently represented between screen transition diagrams and detailed screen specification tables requires Level 2 (Structural Recognition). In contrast, under the “Sufficiency” perspective, checking whether the operational flow defined in business process diagrams is appropriately covered by the screen transition diagram, including conditional branches such as success and failure paths, requires Level 3 (Semantic Recognition).

By combining these review perspectives with corresponding recognition levels, we can delineate the current capabilities and limitations of multimodal LLMs in design document review. Moreover, this framework can also serve as a communication aid in practical deployments, helping end-users understand the feasible scope of automated support.

In this study, we empirically evaluate the feasibility of each level by applying multimodal LLMs to real-world screen design review tasks. The experimental setup and results are presented in the following sections.

IV. PROPOSED METHOD

A. System Architecture and Processing Flow

To effectively conduct LLM-assisted reviews of design documents that include diagrams, our approach involves the following three key processing steps:

- 1) Diagram Region Extraction (Layout Detection)
- 2) Accurate Diagram Understanding (Diagram Understanding)
- 3) Integrated Review with Tables and Text (Multimodal Integration)

Figure 2 illustrates the overall system architecture proposed in this study.

(1) Diagram Region Extraction

First, the entire target design document is rendered as a sequence of page images. To identify regions containing diagrams and tables, we leverage Azure AI Document Intelligence [29], a SaaS-based document analysis service. Since layout analysis itself is not the focus of this study, we outsource

TABLE II
COMPARISON BETWEEN IMAGE-BASED AND OOXML-BASED ANALYSIS

Approach	Pros	Cons
Image-based	• Can visually capture node shapes and layouts. • Can exclude unrelated surrounding info.	• Label recognition errors due to resolution/font/OCR. • Edge connection errors due to ambiguous layout.
OOXML-based	• Accurate extraction of labels. • Precise connection info (when properly authored).	• Includes non-diagram elements. • Incorrect/missing connector references. • Large token size problematic for LLM input.

this task to such an external service. The detected bounding boxes are used to crop the relevant areas, extracting individual diagram images. These cropped images are then passed to the diagram understanding module for further processing.

(2) Accurate Diagram Understanding

Each extracted diagram image is processed by a VLM to obtain an initial understanding. However, as discussed in Section 2, VLMs alone struggle with accurate structural and semantic comprehension of diagrams. To address this, we propose a chain-of-thought (CoT) prompting technique [30] that leverages both the diagram image and auxiliary structural cues derived from the underlying OOXML data, as explained in Section 4.2.

Based on the assumption that recognized diagram structures should be transformed into a domain-specific language (DSL) for downstream processing, we conducted preliminary evaluations with several DSL formats. We ultimately selected PlantUML as the output format, due to its generality, concise syntax, and ease of normalization. The recognized structural information is converted into PlantUML code, which serves as an intermediate representation suitable for subsequent LLM prompting.

(3) Integrated Review with Tables and Text

In addition to diagrams, ensuring consistency with tabular and textual information in the design document is critical for a comprehensive review. In our prior work [5], we proposed a method for converting tabular data into intermediate representations amenable to LLM-based analysis. This study extends that approach by integrating the recognized diagram structures (in PlantUML format) with information extracted from tables and free-text descriptions.

B. Supplementary Structural Recognition via OOXML

In this study, we complement the limitations of image-based diagram recognition by incorporating structural information extracted from Office Open XML (OOXML), the document format used by Microsoft Office applications. OOXML explicitly encodes nodes, edges, labels, and their geometric properties in XML form. When the design document is suitably authored, such information can be extracted and reconstructed into intermediate representations such as JSON.

While our implementation targets OOXML-based diagrams due to their prevalence in Japanese industrial software design documents, the proposed strategy is not inherently tied to OOXML. Similar techniques may be applicable to other visual modeling formats that expose structural metadata—such as

SVG-based tools or domain-specific XML schemas used in CASE tools. Although the quality and granularity of extractable information may vary, the core principle of supplementing visual inputs with structured metadata remains broadly applicable.

From OOXML descriptions, the following types of information can be obtained:

- Nodes: shape types (e.g., rectangles, ellipses), text labels, and positional coordinates (left, top, right, bottom)
- Edges: source and target shape IDs, connection anchors, arrow directions, and auxiliary attributes

We confirmed that these structures can be reliably extracted into JSON and that lightweight structural reconstruction is feasible by connecting nodes and edges based on their identifiers.

However, OOXML-based diagram recognition presents several challenges. Image-based recognition provides advantages such as visually capturing node shapes and layout, and naturally filtering out irrelevant decorations. At the same time, it suffers from limitations including label extraction errors due to resolution or OCR constraints, and insufficient structural recognition accuracy. In contrast, OOXML-based recognition offers precise label extraction and explicit connector information when correctly annotated, yet also introduces the following issues:

- OOXML often contains decorative shapes or annotation text not related to the diagram, requiring additional filtering.
- Connector definitions may be incorrect or incomplete, such as connectors not being properly attached to node shapes.
- OOXML documents are structurally complex, and feeding raw OOXML directly into an LLM is impractical due to token-size constraints.

Table 2 summarizes the advantages and disadvantages of both image-based and OOXML-based approaches. Based on these trade-offs, our method adopts a hybrid strategy that leverages the strengths of both. First, the VLM processes the diagram image to capture visual layout and node placement. Structural information extracted from OOXML is then integrated as a complementary intermediate representation. The combined representation is preserved for downstream processing, enabling more structured and semantically informed reasoning in the subsequent prompting phase.

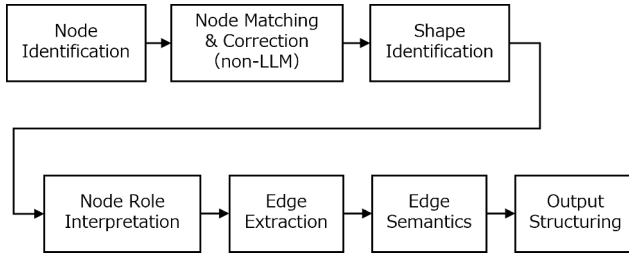


Fig. 3. Overall Chain-of-Thought (CoT) pipeline.

C. CoT-based Diagram Understanding

To transform the diagram information into a structured, LLM-readable format, we employ a Chain-of-Thought (CoT) prompting strategy based on a hybrid input of image-based and OOXML-derived information. The goal of this process is to guide the LLM through a step-by-step reasoning sequence, helping it understand the semantic structure of the diagram with high fidelity.

a) *Reasoning Procedure.*: The CoT prompting process follows the structured flow shown below:

- 1) **Node Identification**: The LLM is first prompted to enumerate all nodes visually present in the diagram. At this stage, no special prompting is required—nodes are listed based on the OCR results of the image-based recognition module.
- 2) **Node Matching & Correction**: This step is performed as a non-LLM preprocessing routine. To mitigate OCR errors in the label text, we align each identified node with OOXML-extracted candidates. Since label lengths are often preserved even with recognition errors, we use edit distance (Levenshtein distance) to select the most likely OOXML node when no exact match is found. This alignment also enables label correction using more reliable OOXML text.
- 3) **Shape Identification**: By combining image-based shape appearance with OOXML-provided shape type and coordinates, we prompt the LLM to determine each node's shape class (e.g., rectangle, ellipse).
- 4) **Node Role Interpretation**: Based on the shape and position of each node, the LLM is instructed to infer its semantic role in the diagram, guided by general domain knowledge from software engineering (e.g., common flowchart constructs like “Start,” “Decision,” or “Process”).
- 5) **Edge Extraction**: The LLM identifies edges connecting the nodes. Here, both image-based edge paths and OOXML connector metadata are considered. Since OOXML connectors are sometimes not attached to node shapes, both visual and structural signals are used to infer proper linkages.
- 6) **Edge Semantics**: Next, the LLM is prompted to assign semantic meanings to each edge (e.g., transition condition, branch direction) based on the source/target nodes, connector type, and any accompanying label text.

- 7) **Output Structuring**: Finally, the structured understanding of the diagram is prepared for the next-stage transformation, where the inferred nodes and edges will be converted into PlantUML. This step enables integration with textual and tabular review targets in markdown format.

This CoT-driven reasoning flow ensures that the LLM incrementally builds semantic understanding while leveraging both image and OOXML-derived information. Rather than attempting one-shot diagram translation, each step decomposes the task into interpretable subtasks that improve explainability and accuracy.

Figure 3 illustrates the overall CoT-based pipeline described above, while Listing 1 presents an actual prompt example used during the edge extraction step, demonstrating how structured inputs are incorporated into LLM-driven reasoning.

Listing 1. Example Prompt for Edge Extraction

You are given a set of nodes extracted from a diagram, with their labels, IDs, and shapes, along with connector information derived from the OOXML representation.

Nodes:

- Node A (ID: n1): “Start” — Shape: Ellipse
- Node B (ID: n2): “Check Status” — Shape: Diamond
- Node C (ID: n3): “Request” — Shape: Rectangle
- Node D (ID: n4): “Request” — Shape: Rectangle

Connectors (from OOXML):

- Connector 1: from n1 to n2, arrow: yes
- Connector 2: from n2 to n3, arrow: yes
- Connector 3: from n2 to n4, arrow: yes

Note: OCR-based recognition may include label noise. Use both the OOXML connector data and the visual layout of the diagram (e.g., spatial positions) to infer the edges between nodes. Prioritize OOXML when connectors are present and valid.

Your task: Infer the set of directed edges between nodes.

Output format: A list of JSON objects. Each object must include:

- source_id
- target_id

While this method is implemented using OOXML-derived structure in this study, the overall procedure may also be applicable to other sources of diagram structure—such as vector graphics (e.g., SVG), business modeling notations (e.g., BPMN XML), or DSL-generated models—depending on the level of semantic and geometric information available. The effectiveness would vary depending on the richness and fidelity of the structured representation, but the modular nature of the reasoning process allows adaptation with minimal changes to the pipeline.

V. EXPERIMENTAL EVALUATION

A. Research Questions

This section evaluates the effectiveness of the proposed hybrid diagram-understanding method using real-world software design documents. Our evaluation is designed to answer the following research questions:

- **RQ1:** Does the proposed method enable LLMs to accurately recognize diagrams contained in real design documents?
- **RQ2:** To what extent can the proposed method support practical review tasks in software design?

B. Experimental Environment

All experiments were conducted using the o3 model (2025-04-16) provided via Microsoft Azure OpenAI Service. Since the design documents used in this study originate from an actual financial-domain system, strict security considerations were required. To avoid any risk of data leakage, all evaluations were executed exclusively within the Azure-hosted GPT environment under our enterprise contract. The model was accessed via the Chat Completions API with default settings, and no external plugins or tools were used. Prompt templates were standardized across runs to ensure consistency, and all outputs were postprocessed to normalize minor format differences before evaluation.

C. Evaluation Tasks and Datasets

RQ1: Evaluation of Diagram Recognition Accuracy

To answer RQ1, we evaluated whether the proposed hybrid method can accurately recognize real-world diagrams across three dimensions: (1) node identification, (2) edge identification, and (3) edge semantics (transition conditions). We used two types of diagram sources:

- Financial-domain design documents: 24 diagrams were taken from actual design documents created for enterprise banking systems at Fujitsu. These documents were originally written in Japanese and used in real-world projects. The diagrams include 3 screen hierarchy diagrams, 12 screen transition diagrams, and 9 business process flowcharts. Figure 4 presents an example business process diagram used in our evaluation (all sensitive information has been masked for privacy).
- FlowLearn dataset [26]: 16 non-handwritten diagrams selected from the FlowLearn benchmark, excluding sketch-like or highly stylized samples. Since structural information is not provided in this dataset, we manually reconstructed each diagram using Microsoft Excel to create OOXML-based representations that simulate structured design documents. During this reconstruction, all connectors were carefully attached to the corresponding node shapes exactly as shown in the original diagram, ensuring structural consistency. An example is presented in Figure 5.

For each diagram, we manually prepared gold-standard annotations. The LLM outputs were compared against these

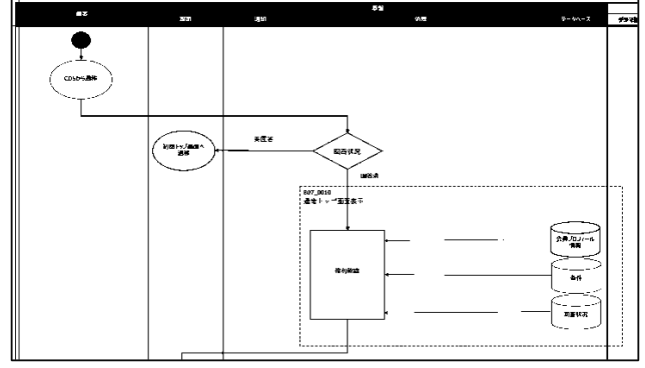


Fig. 4. Example business process diagram from a financial-domain design document

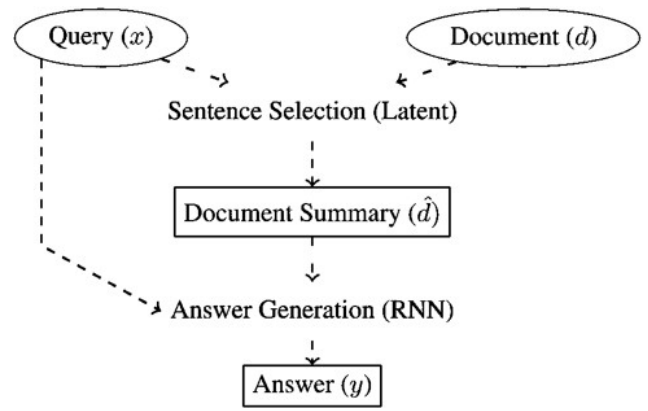


Fig. 5. Example diagram from FlowLearn

annotations to compute precision and recall for nodes, edges, and the semantics of inter-node transitions.

RQ2: Evaluation of Applicability to Review Tasks

To answer RQ2, we investigated how far the proposed method can support realistic design-review tasks. We selected two review perspectives from Table 1, corresponding to different levels of diagram-recognition difficulty:

- 1) Consistency Check (Structural Recognition Level): Whether the content of a screen transition diagram is consistent with the corresponding detailed screen specification.
- 2) Sufficiency Check (Semantic Recognition Level): Whether the operational flows defined in a business process diagram are fully realized in the corresponding screen transition diagram, including transition conditions.

To better isolate the impact of diagram recognition quality, both the consistency and sufficiency checks were conducted under two input conditions. In the first condition, diagrams were generated by the proposed hybrid recognition method evaluated in RQ1, and thus may contain recognition errors.

TABLE III
RECOGNITION ACCURACY FOR RQ1 (PRECISION AND RECALL).

	Node		Edge		Transition Condition	
	Prec.	Rec.	Prec.	Rec.	Prec.	Rec.
Financial-domain documents	0.96	0.92	0.75	0.83	0.35	0.45
FlowLearn dataset	0.96	0.96	0.92	1.00	0.65	0.62

In the second condition, diagrams were manually transcribed into accurate PlantUML descriptions, representing an ideal scenario of perfect diagram understanding. This comparative setup allowed us to assess how recognition quality affects downstream review performance for both structural (consistency) and semantic (sufficiency) review tasks.

(1) Consistency Check Evaluation.

We adopted a “bidirectional” consistency-checking procedure, commonly used in industrial design reviews [31]. For each experiment, one detailed screen specification and two variants of a screen transition diagram were used. Intentional inconsistencies were inserted into the screen transition diagram by replacing the correct screen identifier with a different, valid identifier (e.g., changing “SC_A02_0010” to “SC_A03_0010”). The evaluation measured whether the LLM could detect such cross-document discrepancies. Precision and recall were computed across 12 screen transition diagrams.

(2) Sufficiency Check Evaluation.

For sufficiency, we evaluated whether the business-process-defined flows were fully reflected in the screen transition diagrams. The LLM was required to compare:

- nodes in the business flow vs. screen transition diagram,
- structural transitions between nodes,
- transition conditions.

Intentional defects were introduced into the screen transition diagrams, limited to omissions or modifications of transition conditions. For example, although the business-flow diagram specifies a conditional transition from “SC_A02_0010” to “SC_A03_0010”, the corresponding condition was intentionally removed or altered in the screen transition diagram. Precision and recall were computed based on whether the LLM correctly detected each missing or modified condition.

The evaluation was conducted using 6 business process diagrams and their corresponding 6 screen transition diagrams, all sourced from real-world financial-domain design documents. Each pair was manually aligned in advance to ensure semantic correspondence before injecting the intentional defects.

D. Experimental Results for RQ1

Table III summarizes the recognition accuracy of the proposed hybrid method across three dimensions: node identification, edge identification, and the semantics of transition conditions.

Node recognition achieved more than 90% precision and recall in both datasets, confirming that the proposed method can reliably extract screen identifiers and processing steps from real design artifacts. Edge recognition also showed favorable

```
@startuml
skinparam linetype ortho

ellipse "Query (x)" as Query
ellipse "Document (d)" as Document
rectangle "Sentence Selection¥n(Latent)" as Sentence
rectangle "Document Summary¥n(d^)" as Summary
rectangle "Answer Generation¥n(RNN)" as Generation
rectangle "Answer (y)" as Answer

Query -right-> Sentence : dashed, blue
Document -left-> Sentence : dashed, blue
Sentence -down-> Summary : dashed
Summary -down-> Generation
Generation -down-> Answer
Query -down-> Generation : dashed, blue

@enduml
```

Fig. 6. Inferred PlantUML from Figure 5.

performance, with recall exceeding 80% for financial-domain diagrams and 100% for FlowLearn samples. These results indicate that the structural aspects of diagrams are largely captured correctly. An example conversion of the diagram in Figure 5 into PlantUML, based on the recognized nodes and edges, is shown in Figure 6.

The perfect recall in the FlowLearn dataset can be attributed to the fact that we manually reconstructed each diagram in Excel, ensuring that all connectors were properly attached to node shapes, thereby providing fully structured OOXML representations. In contrast, some of the financial-domain diagrams exhibited missing or incomplete connector information even in their OOXML sources. This resulted in occasional edge omissions. However, we observed that the proposed method could sometimes infer missing edges by combining node shape positioning and image-based path cues, even when the structural connector was absent in the OOXML data. This highlights the value of our hybrid approach in compensating for real-world structural noise or authoring inconsistencies.

However, transition-condition recognition exhibited substantially lower accuracy, particularly for financial-domain diagrams, where precision dropped to 0.35 and recall to 0.45. A detailed error analysis revealed that many failures were caused by complex, multi-clause conditions commonly found in real banking workflows. For example, in the screen transition diagram shown in Figure 7, the LLM failed to correctly interpret nested conditional expressions involving multiple branches and

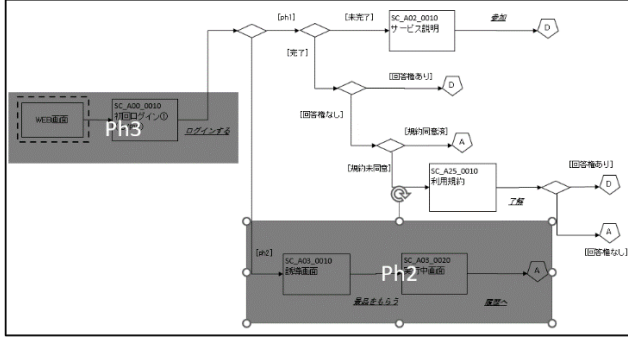


Fig. 7. Recognition failure case involving nested transition logic.

exception-handling flows. Such cases frequently resulted in partial extraction, incorrect decomposition of logical operators, or hallucinated condition fragments.

We additionally explored several prompt-engineering strategies, including providing few-shot examples of multi-step conditional transitions. As part of this effort, we also introduced an intermediate CoT step in which the LLM was explicitly prompted to infer transition conditions before assigning semantic labels to edges. This modular approach showed some improvement in handling complex, multi-branch transitions. However, it also introduced side effects: in simpler cases where no multi-step conditions existed in the image, the model occasionally over-inferred transitions, resulting in hallucinated logic. Despite these interventions, we did not observe a consistent or substantial gain in overall accuracy. These results suggest that prompt engineering alone may not be sufficient to robustly handle the semantic intricacies of transition conditions, especially in domains with implicit or context-sensitive logic. More principled integration of external knowledge or symbolic reasoning mechanisms may be needed to overcome these limitations in future work.

Answer to RQ1: The proposed hybrid method enables large language models to accurately recognize structural elements in real-world design diagrams, achieving over 90% precision and recall for node identification and strong performance for edge extraction. However, it falls short in reliably interpreting complex transition conditions, particularly in financial-domain diagrams, indicating that semantic-level understanding remains a significant challenge for current-generation multimodal LLMs.

E. Experimental Results for RQ2

(1) Consistency Check (Structural Level).

Table IV summarizes the results of the consistency-check task, which focused on structural discrepancies between screen

transition diagrams and detailed screen specifications. Under both input conditions, (1) Perfect PlantUML and (2) diagrams recognized by the proposed method, the model achieved similarly high precision (0.96), while recall showed a modest drop from 0.95 to 0.90 under the recognition-based condition. This suggests that the proposed method preserves most of the structural fidelity necessary for detecting inconsistencies, although a small number of screen IDs or nodes may be missed during recognition.

Closer inspection of false negatives revealed that most of the missed inconsistencies stemmed from cases where the screen identifier in the recognized diagram was either slightly misread or entirely omitted due to edge- or node-recognition failure. Overall, these results indicate that the hybrid recognition pipeline provides sufficient structural accuracy to support practical consistency-checking tasks, while also highlighting the potential benefits of minor refinement steps to recover missed identifiers or improve robustness against imperfect diagram inputs.

(2) Sufficiency Check (Semantic Level).

In the sufficiency-check task, the LLM was required to detect missing or incorrect transition conditions by comparing business process diagrams with screen transition diagrams. As shown in Table V, performance varied substantially depending on the input condition.

When provided with accurate diagram inputs via manually constructed PlantUML, the model achieved strong performance (0.92 precision, 0.88 recall), demonstrating its potential for handling semantic-level comparisons when given clean inputs. However, performance dropped significantly to 0.26 precision and 0.28 recall under the recognition-based condition, highlighting the limitations of current recognition accuracy for high-level review tasks.

Manual inspection showed that many failures stemmed from the model's difficulty in parsing complex or nested conditional expressions. These expressions, which often govern the logic of screen transitions, were frequently omitted or misinterpreted, leading to missed detections.

These results indicate that semantic-level tasks such as sufficiency checking remain challenging with current recognition capabilities. While structural aspects are reasonably well preserved, more advanced handling of logical expressions and domain-specific semantics is needed to reliably support meaning-intensive review scenarios.

(3) Estimation of Practical Impact in Real-World Review Tasks.

While the proposed method demonstrates sufficient accuracy for structural-level checks (Level 2), it does not yet offer reliable performance at the semantic level (Level 3). To estimate the practical utility of current capabilities, we analyzed actual design-review checklists from three enterprise software packages developed and used within Fujitsu, spanning the banking, apparel, and real estate domains.

Each package differs in the types of diagrams it emphasizes: the banking system includes many screen transition diagrams, the apparel system centers around business process flows,

TABLE IV
CONSISTENCY CHECK RESULTS (STRUCTURAL-LEVEL TASK)

Input Type	Error Detection Precision	Error Detection Recall
Perfect PlantUML	0.96	0.95
LLM Recognition	0.96	0.90

TABLE V
SUFFICIENCY CHECK RESULTS (SEMANTIC-LEVEL TASK)

Input Type	Error Detection Precision	Error Detection Recall
Perfect PlantUML	0.92	0.88
LLM Recognition	0.26	0.28

and the real estate system represents a balanced combination. Based on a classification of checklist items by recognition level, we found that the proportion of review items corresponding to Level 1 and Level 2 checks was highest in the banking package (42%), lowest in the apparel package (23%), and 31% when calculated across all three packages combined.

Although this indicates that roughly one-third of diagram-related review items could potentially benefit from automation, the actual reduction in review effort is likely to be lower. This is because semantic-level checks (Level 3), which the current method does not yet support reliably, typically require substantially more reviewer time and expertise per item than Levels 1 and 2. As a result, even full automation of structural checks may only translate into a modest decrease in total review effort. Nonetheless, these findings suggest that the current method can already contribute meaningful support, especially in domains like banking where structural checks are prevalent.

Answer to RQ2: The proposed method can effectively support design-review tasks that are primarily structural in nature, such as many aspects of consistency checking, achieving high error detection performance even when using automatically recognized diagrams. In contrast, tasks that involve more complex semantic reasoning—especially those requiring precise understanding of transition conditions, as in parts of sufficiency checking—remain more challenging. The results suggest that while structural review tasks are already well-supported, further improvements in semantic recognition are needed to fully address meaning-intensive scenarios.

VI. CONCLUSION

Building on our prior work that explored LLM-based automated reviews for textual design documents, this study focused specifically on extending review capabilities to include diagrammatic information, such as screen transition diagrams and business process flows. We proposed a hybrid method that combines vision-based recognition with structured metadata

extraction, enabling multimodal LLMs to analyze diagrams embedded in real-world design artifacts.

Our evaluation demonstrated that the method is effective in supporting structurally oriented review tasks, such as many consistency checks, even when using automatically recognized diagrams. However, tasks requiring precise semantic interpretation proved more challenging, and recognition errors substantially degraded downstream review performance. These findings underscore the importance of accurate semantic recognition for applying LLMs to meaning intensive review tasks.

While our method makes progress toward multimodal automated reviews, several limitations remain. First, the current approach targets only a limited set of diagram types—primarily screen transition diagrams and process flowcharts. It does not yet handle other forms of visual information common in design documents, such as screen layout diagrams or annotated UI mockups.

Future research directions in the context of multimodal review tasks include:

- **Expansion to Other Visual Representations:** Extending applicability to diverse visual artifacts beyond diagrams, including wireframes, screen mockups, and layout illustrations commonly found in design documents.
- **Improved Semantic Recognition:** Enhancing the model’s ability to interpret complex transition conditions and nested logic across modalities.
- **Adaptive Prompting and Interaction:** Creating prompting strategies that adjust based on diagram type and review intent.

As multimodal LLMs continue to improve, we believe that combining structured metadata, adaptive prompting, and task-specific grounding will enable more comprehensive and reliable design document reviews, encompassing both textual and visual elements within future enterprise systems. In this view, diagram understanding is not an isolated task but an essential component of a comprehensive, multimodal review strategy. Future enterprise review systems may thus benefit from combining this diagram-centric layer with existing textual review agents to provide more reliable and context-aware assessments across design artifacts.

REFERENCES

- [1] Priyan Vaithilingam, Tianyi Zhang, Elena L. Glassman, Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In CHI Conference on Human Factors in Computing Systems Extended Abstracts. ACM, NY NY, USA, pp. 1–7 (2022).
- [2] Perry, D.: Where do Most Software Flaws Come from?, Making Software: What Really Works, and Why We Believe It (Oram, A. and Wilson, G., eds.). O'Reilly Media, Chapter 25, pp. 453–494 (2010).
- [3] Takasaburo Fukuda, Kentaro Murase, Satoru Nakamura, Kazuo Hiekata, Yoshiaki Oida, Shigeru Matsumoto, Isaku Okada, Development of Impact Analysis System for Specification Change of System with Design Information and Process Dependency. Journal of Information Processing, vol. 59, no. 4, pp. 1240–1249 (2018).
- [4] Atsushi Motoyama, Improvement in quality of design specifications by detecting inconsistencies between design items. ProMAC 2013 (2013).
- [5] Takasaburo Fukuda, Takao Nakagawa, Keisuke Miyazaki, and Susumu Tokumoto, Development of Automated Software Design Document Review Methods Using Large Language Models. 2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 91–101 (2025).
- [6] NTT Com Research and President Inc. Joint Study. "Survey on Document Creation by Business People." NTT Com Research Database. Available at: <https://research.nttcoms.com/database/data/001928/> (accessed 2025).
- [7] ISO/IEC 29500: Information technology – Document description and processing languages – Office Open XML File Formats. International Organization for Standardization (2008). Available at: <https://www.iso.org/standard/51463.html> (accessed 2025).
- [8] IEEE. 1028–2008 IEEE standard for software reviews and audits. (2008).
- [9] ISO/IEC 20246:2017 Software and systems engineering – Work product reviews (2017).
- [10] Yuri Chernak, A Statistical Approach To The Inspection Checklist Formal Synthesis And Improvement. IEEE Transactions on Software Engineering, 22(12):866–874 (1996).
- [11] Victor R. Basili, Evolving And Packaging Reading Technologies. Journal of Systems and Software 38(1), 3–12 (1997).
- [12] Information-technology Promotion Agency, Japan (IPA), Software Development Analysis Data Collection 2022 (Financial and Insurance Industries Edition) (2022).
- [13] Larissa Schmid, Tobias Hey, Martin Armbruster, Sophie Corallo, Dominik Fuchß, Jan Keim, Haoyu Liu, and Anne Koziol. Software Architecture Meets LLMs: A Systematic Literature Review. arXiv preprint arXiv:2505.16697 (2025).
- [14] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang, Large language models for software engineering: A systematic literature review. arXiv preprint arXiv:2308.10620 (2023).
- [15] Madhava Krishna, Bhagesh Gaur, Arsh Verma, Pankaj Jalote. Using LLMs in Software Requirements Specifications: An Empirical Evaluation. arXiv preprint arXiv:2404.17842 (2024).
- [16] Pouria Rouzrokh, Bardia Khosravi, Parsa Rouzrokh, and Moein Shariatnia. LatteReview: A Multi-Agent Framework for Systematic Review Automation Using Large Language Models. arXiv preprint arXiv:2501.05468 (2025).
- [17] Xian Gao, Jiacheng Ruan, Zongyun Zhang, Jingsheng Gao, Ting Liu, and Yuzhuo Fu. MMReview: A Multidisciplinary and Multimodal Benchmark for LLM-Based Peer Review Automation. arXiv preprint arXiv:2508.14146 (2025).
- [18] Sirui Hong and Xiaowu Zheng and Jonathan Chen and Yuheng Cheng and Jinlin Wang and Ceyao Zhang and Zili Wang and Steven Ka Shing Yau and Zijuan Lin and Liyang Zhou and Chenyu Ran and Lingfeng Xiao and Chenglin Wu, Metagpt: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, (2023).
- [19] Javier Cámara, Javier Troya, Lola Burgueño, and Antonio Vallecillo. On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML. Softw. Syst. Model. 22, 3, 781–793, (2023).
- [20] Hideyuki Kanuka, Genta Koreki, Ryo Soga, Kazu Nishikawa, An experiment for applying ChatGPT to bidirectional traceability problem between design model and code. Software Engineering Symposium 2023 (SES2023), 238–239, (2023).
- [21] Yifan Hou, Buse Giledereli, Yilei Tu, and Mrinmaya Sachan. Do Vision-Language Models Really Understand Visual Language? arXiv preprint arXiv:2410.00193 (2025).
- [22] Ryo Kamoi, Yusen Zhang, Sarkar Snigdha Sarathi Das, Ranran Haoran Zhang, and Rui Zhang. VisOnlyQA: Large Vision Language Models Still Struggle with Visual Perception of Geometric Information. arXiv preprint arXiv:2412.00947 (2024).
- [23] Yichen Xu, Liangyu Chen, Liang Zhang, Wenxuan Wang, and Qin Jin. POLYCHARTQA: Benchmarking Large Vision-Language Models with Multilingual Chart Question Answering. arXiv preprint arXiv:2507.11939 (2025).
- [24] Gábor Antal, Richárd Vozár, and Rudolf Ferenc. Assessing GPT-4-Vision's Capabilities in UML-Based Code Generation. arXiv preprint arXiv:2404.14370 (2024).
- [25] Mengliang He, Jiayi Zeng, Yankai Jiang, Wei Zhang, Zeming Liu, Xiaoming Shi, and Aimin Zhou. Flow2Code: Evaluating Large Language Models for Flowchart-based Code Generation Capability. arXiv preprint arXiv:2506.02073 (2025).
- [26] Huitong Pan, Qi Zhang, Cornelia Caragea, Eduard Dragut, and Longin Jan Latecki. FlowLearn: Evaluating Large Vision-Language Models on Flowchart Understanding. arXiv preprint arXiv:2407.05183 (2024).
- [27] Shue Shiinoki, Ryo Koshihara, Hayato Motegi, and Masumi Morishige. Overcoming Vision Language Model Challenges in Diagram Understanding: A Proof-of-Concept with XML-Driven Large Language Models Solutions. arXiv preprint arXiv:2502.04389 (2025).
- [28] Hiroshi Sasaki. Structure-aware Contrastive Learning for Diagram Understanding of Multimodal Models. arXiv preprint arXiv:2509.01959 (2025).
- [29] Microsoft. Azure AI Document Intelligence. Available at: <https://learn.microsoft.com/en-us/azure/ai-services/document-intelligence/> (accessed 2025).
- [30] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, Denny Zhou, Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems (2022).
- [31] Atsushi Motoyama, Takako Nakatani, A metamodel for design review derived from design specification templates. JCKBSE 2012, 108–117 (2012).